UNIVERSITÉ LAVAL

Solutionnaire du TP2

Partie A : Game of Thrones

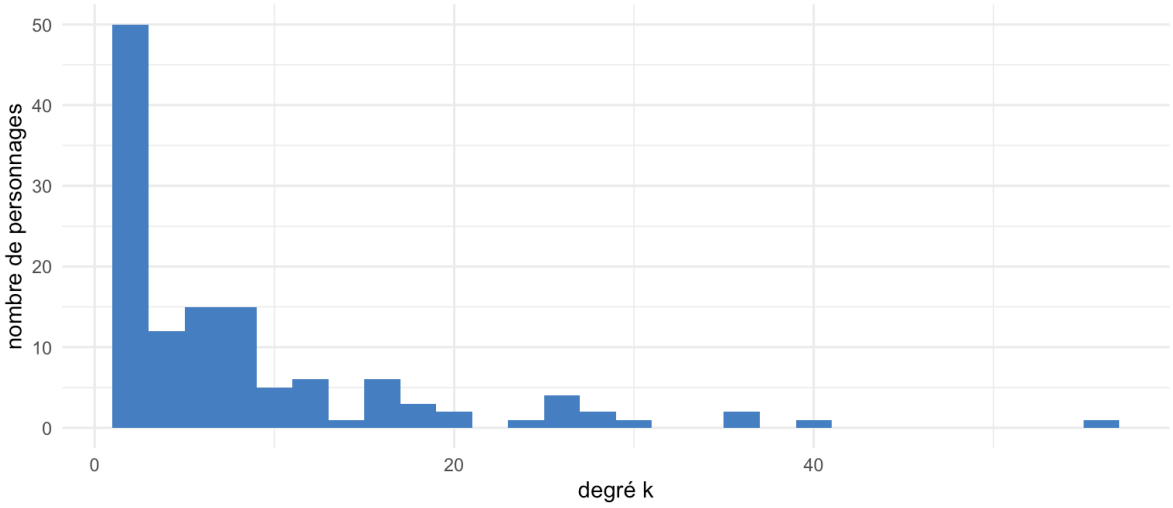
0. Reconstruire le graphe

```
got_aretes <- read_csv("/home/public/reseaux/got_edges.csv")
got_sexe   <- read_csv("/home/public/reseaux/got_sex.csv")
```

```
got <- graph_from_data_frame(got_aretes, directed = FALSE, vertices = got_sexe)
cat(capture.output(print(got))[1:2], sep = "\n")
#> IGRAPH b3ab3dd UNW- 127 550 --
#> + attr: name (v/c), sex (v/c), weight (e/n)
```

1. Degré et force

```
tibble(k = degree(got)) |>
  ggplot(aes(k)) + geom_histogram(binwidth = 2, fill = "#4A7FC1") +
  labs(x = "degré k", y = "nombre de personnages")
```




```
sum(degree(got) <= 3)
#> [1] 50
sort(degree(got), decreasing = TRUE)[1:5]
```

```
#>     Ned  Tyrion Catelyn  Robert    Robb
#>      57      41      37      36      30
sort(strength(got), decreasing = TRUE)[1:5]
#>      Ned   Tyrion  Catelyn   Robert Daenerys
#>     1290      708      584      563      535
```

Une distribution très asymétrique : 50 personnages sur 127 ont au plus 3 liens, quelques-uns en ont des dizaines (une « queue lourde »). Les quatre premiers sont les mêmes pour le degré et la force ; Robb (30 liens) est remplacé par Daenerys en force : elle a moins de partenaires (18), mais interagit **beaucoup** avec eux (Jorah, Drogo…).

2. Intermédiarité : avec ou sans poids

```
sort(betweenness(got, weights = NA), decreasing = TRUE)[1:5]
#>       Ned    Tyrion   Catelyn    Robert  Daenerys
#> 2364.5574 1271.4956 1041.7014  860.8102  798.4258
sort(betweenness(got), decreasing = TRUE)[1:5]
#>       Ned     Jorah     Aerys   Catelyn    Tyrion
#> 1599.5311 1220.4734  796.5135  778.6164  767.3571
```

Sans préciser `weights`, igraph utilise l'attribut `weight` comme une **longueur** : un lien de 192 interactions devient un « long » chemin. Jorah et Aerys entrent dans le top 5, Robert et Daenerys en sortent. Ici le poids mesure une **proximité** : seul le classement avec `weights = NA` a un sens. Ned est en tête dans les deux cas, mais ce n'est pas une garantie.

3. Chemins et diamètre

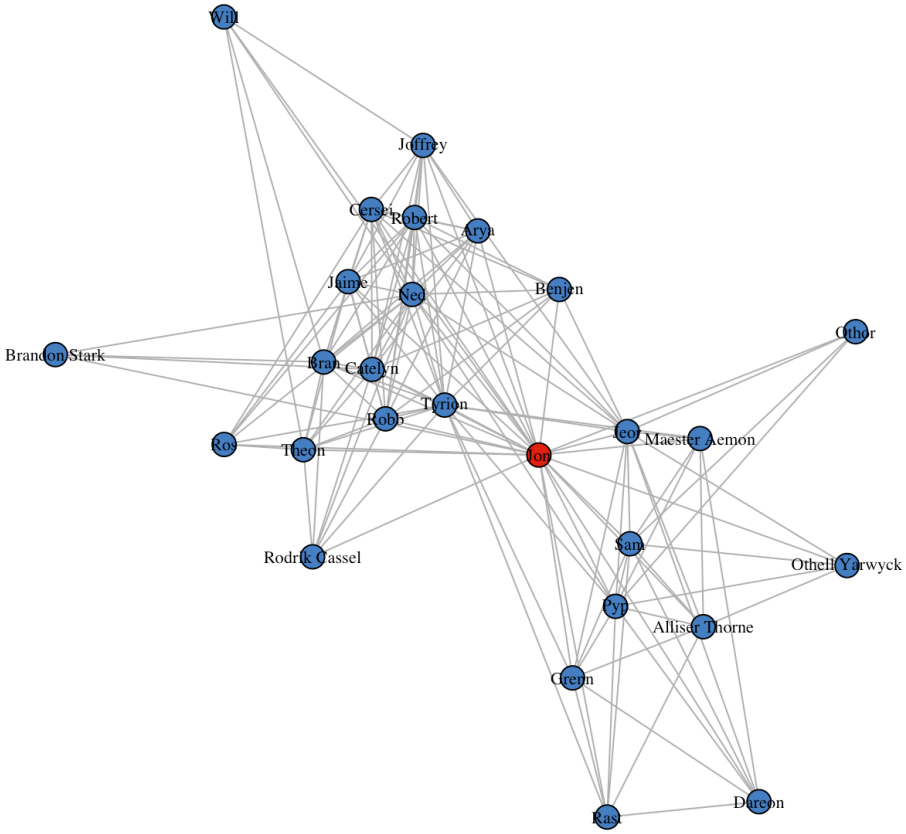
```
distances(got, "Jon", "Daenerys", weights = NA)
#>     Daenerys
#> Jon        2
shortest_paths(got, "Jon", "Daenerys", weights = NA)$vpath[[1]]
#> + 3/127 vertices, named, from b3ab3dd:
#> [1] Jon      Ned      Daenerys
diameter(got, weights = NA)
#> [1] 6
get_diameter(got, weights = NA)
#> + 7/127 vertices, named, from b3ab3dd:
#> [1] Melessa  Sam      Jon      Ned      Daenerys Doreah   Irogenia
```

Jon et Daenerys, que tout sépare dans la saison 1, ne sont qu'à 2 liens, par Ned. Le diamètre est de 6 : aucun personnage n'est à plus de 6 étapes d'un autre (un petit monde). `get_diameter()` renvoie **un** des chemins de cette longueur ; il peut y en avoir plusieurs.

4. Le réseau ego de Jon

```
ego_jon <- make_ego_graph(got, order = 1, nodes = "Jon")[[1]]
vcount(ego_jon)
```

```
#> [1] 27
ecount(ego_jon)
#> [1] 143
degree(got, "Jon")
#> Jon
#>  26
set.seed(1)
par(mar = c(0, 0, 0, 0))
plot(ego_jon, vertex.size = 6, vertex.label.cex = 0.7, vertex.label.color = "black",
     vertex.color = if_else(V(ego_jon)$name == "Jon", "#E30513", "#4A7FC1"), edge.color =
```



`make_ego_graph()` renvoie une **liste** de graphes (un par nœud demandé) : d'où le `[[1]]`. Le réseau ego contient Jon et ses 26 voisins, soit 27 nœuds, et **tous** les liens entre eux : 143 liens, dont 117 entre voisins de Jon.

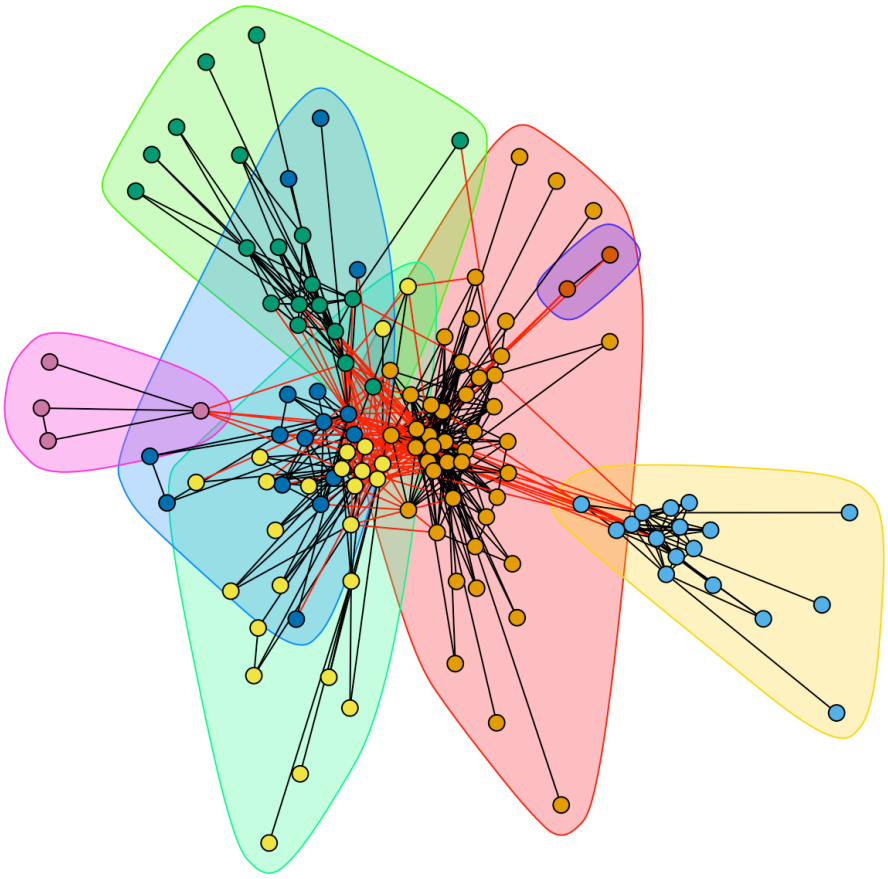
5. Les cliques

```
clique_num(got)
#> [1] 10
cliques_max <- largest_cliques(got)
length(cliques_max)
#> [1] 7
Reduce(intersect, lapply(cliques_max, names))
#> [1] "Jaime"   "Ned"     "Robert"  "Cersei"  "Catelyn" "Arya"
```

7 cliques de 10 personnages, qui partagent 6 membres : ce sont presque les mêmes. Une clique est trop stricte pour définir des groupes : un seul lien manquant suffit à la casser.

6. Les communautés

```
set.seed(1)
comm <- cluster_louvain(got)
length(comm)
#> [1] 7
sizes(comm)
#> Community sizes
#>  1  2  3  4  5  6  7
#> 45 17 19 24 16  2  4
modularity(comm)
#> [1] 0.5300541
membership(comm)[c("Ned", "Daenerys")]
#>      Ned Daenerys
#>        1        2
names(which(membership(comm) == membership(comm)["Daenerys"]))
#>  [1] "Daenerys"       "Jorah"          "Doreah"         "Drogo"
#>  [5] "Illyrio"        "Mirri Maz Duur" "Irri"           "Cohollo"
#>  [9] "Qotho"          "Rakharo"        "Rhaego"         "Viserys"
#> [13] "Wine Merchant"  "Mago"           "Jhiqui"         "Irogenia"
#> [17] "Little Bird"
set.seed(1)
par(mar = c(0, 0, 0, 0))
plot(comm, got, vertex.label = NA, vertex.size = 4)
```



7 communautés, modularité 0.53 (au-dessus de 0,3 : une structure nette). Ned et Daenerys sont dans des communautés **différentes** ; celle de Daenerys réunit les personnages de l'intrigue d'Essos (Jorah, Drogo, Viserys, Illyrio, les Dothrakis). Les numéros sont des **étiquettes** : ils changeront avec une autre graine.

Partie B : un réseau de co-expression

7. De l'expression au graphe

```
coexpr <- read_tsv("/home/public/reseaux/airway_coexpr.tsv")
```

```
m <- coexpr |>
  dplyr::select(symbole, ends_with("trt")) |>
  column_to_rownames("symbole") |>
  as.matrix()
dim(m)
#> [1] 150   8
r <- cor(t(m))
dim(r)
#> [1] 150 150
adj <- (r > 0.8) * 1
diag(adj) <- 0
```

```
g_co <- graph_from_adjacency_matrix(adj, mode = "undirected")
V(g_co)$statut <- coexpr$statut
```

`cor()` calcule les corrélations entre **colonnes** : sans `t()`, on obtiendrait une matrice 8 × 8 de corrélations entre échantillons. `ends_with("trt")` attrape `_untrt` et `_trt`. On peut ajouter `statut` dans l'ordre de `coexpr`, parce que les nœuds suivent l'ordre des lignes de `m`, lui-même celui de `coexpr` :

```
identical(V(g_co)$name, coexpr$symbole)
#> [1] TRUE
```

8. Nœuds, liens, composantes

```
vcount(g_co)
#> [1] 150
ecount(g_co)
#> [1] 1622
sum(degree(g_co) == 0)
#> [1] 3
comp <- igraph::components(g_co)
comp$no
#> [1] 5
sort(comp$csize, decreasing = TRUE)
#> [1] 138   9   1   1   1
```

1622 liens, 3 gènes isolés (chacun est une composante à lui seul), et une composante géante de 138 gènes. Avec `graph_from_adjacency_matrix()`, les isolés restent dans le graphe ; avec une liste d'arêtes, ils auraient disparu (TP1).

9. Les hubs

```
hubs <- sort(degree(g_co), decreasing = TRUE)[1:6]
hubs
#>     DUSP1    ERRFI1      NEXN      TJP2 LINC00968      GPX3
#>        54        54        54        54        54        53
V(g_co)$statut[match(names(hubs), V(g_co)$name)]
#> [1] "induit" "induit" "induit" "induit" "induit" "induit"
table(V(g_co)$statut)
#>
#>  induit  non DE réprimé
#>      78      47      25
```

Les six hubs sont **induits** par la dexaméthasone, dont *DUSP1*, un gène cible classique des glucocorticoïdes. Plusieurs sont à égalité (54 liens) : un bloc de gènes très corrélés entre eux, plutôt qu'un « régulateur maître ». Un hub de co-expression n'est pas un régulateur : la corrélation n'a pas de sens.

10. Les modules

```
set.seed(1)
modules <- cluster_louvain(g_co)
length(modules)
#> [1] 9
sizes(modules)
#> Community sizes
#>  1  2  3  4  5  6  7  8  9
#>  9 18 36 45 29 10  1  1  1
table(module = membership(modules), statut = V(g_co)$statut)
#>       statut
#> module induit non DE réprimé
#>      1      0      9       0
#>      2      0      1      17
#>      3      6     26       4
#>      4     45      0       0
#>      5     26      3       0
#>      6      1      6       3
#>      7      0      0       1
#>      8      0      1       0
#>      9      0      1       0
table(module = membership(modules), chromosome_Y = coexpr$chromosome == "Y", useNA = "ifa
#>       chromosome_Y
#> module FALSE TRUE <NA>
#>      1     0    7    2
#>      2    18    0    0
#>      3    35    0    1
#>      4    44    0    1
#>      5    29    0    0
#>      6    10    0    0
#>      7     1    0    0
#>      8     1    0    0
#>      9     1    0    0
```

```
mod_y <- unique(membership(modules)[which(coexpr$chromosome == "Y")])
mod_y
#> [1] 1
```

La plupart des modules sont **purs** : des induits d'un côté, des réprimés de l'autre (le réseau ne garde que les corrélations **positives**). Les modules d'un seul gène sont les gènes isolés. Le module 1 est fait de gènes **non DE** du chromosome **Y**.

11. Le module du chromosome Y

```
coexpr |>
  filter(membership(modules) == mod_y) |>
  dplyr::select(symbole, chromosome, statut)
```

```
#> # A tibble: 9 × 3
#>   symbole         chromosome statut
#>   <chr>           <chr>      <chr>
#> 1 RPS4Y1          Y          non DE
#> 2 DDX3Y           Y          non DE
#> 3 USP9Y           Y          non DE
#> 4 KDM5D           Y          non DE
#> 5 ENSG00000131002 <NA>       non DE
#> 6 ENSG00000184674 <NA>       non DE
#> 7 NLGN4Y          Y          non DE
#> 8 ZFY             Y          non DE
#> 9 UTY             Y          non DE
m[c("RPS4Y1", "DDX3Y", "XIST"), ]
#>        N61311_untrt N61311_trt N052611_untrt N052611_trt N080611_untrt
#> RPS4Y1       12.272     12.243        12.039      11.952        12.155
#> DDX3Y        10.713     10.613        10.421      10.556        10.761
#> XIST          6.734      6.943         6.916       6.734         6.734
#>        N080611_trt N061011_untrt N061011_trt
#> RPS4Y1      11.991         6.734       6.938
#> DDX3Y       10.882         6.734       6.938
#> XIST         6.734        12.128      11.744
```

Les gènes du Y sont éteints dans les deux échantillons du donneur N061011, qui exprime fortement *XIST* (l'ARN qui inactive un des deux X chez la femme) : c'est une **donneuse**. Les 2 gènes au chromosome `NA` sont des gènes sans symbole qui suivent exactement le même profil (vraisemblablement d'autres gènes du Y). Ce module est parfaitement cohérent et n'a **rien à voir** avec la dexaméthasone : traité ou non, chaque donneur garde son sexe. *XIST*, anticorrélé aux gènes du Y, n'est pas dans le module.

Et si… ?

1. La valeur absolue

```
adj_abs <- (abs(r) > 0.8) * 1
diag(adj_abs) <- 0
g_abs <- graph_from_adjacency_matrix(adj_abs, mode = "undirected")
ecount(g_abs)
#> [1] 2291
set.seed(1)
mod_abs <- membership(cluster_louvain(g_abs))
table(module = mod_abs, statut = coexpr$statut)
#>       statut
#> module induit non DE réprimé
#>      1      0     10       1
#>      2      7     34       5
#>      3     45      0      10
#>      4     26      3       8
#>      5      0      0       1
```

```
mod_abs[c("XIST", "RPS4Y1")]
#>   XIST RPS4Y1
#>      1      1
```

2291 liens au lieu de 1622 : les paires **anticorrélées** sont maintenant liées. Des gènes induits et réprimés se retrouvent dans les mêmes modules (et *XIST* rejoint les gènes du Y). Un réseau « non signé » n'est pas faux, mais un module n'y veut plus dire « varient dans le même sens ».

2. Un seuil plus strict

```
adj_09 <- (r > 0.9) * 1
diag(adj_09) <- 0
g_09 <- graph_from_adjacency_matrix(adj_09, mode = "undirected")
c(liens = ecount(g_09), isoles = sum(degree(g_09) == 0), composantes = igraph::components
#>       liens      isoles composantes
#>        846          23          31
```

846 liens au lieu de 1622, 23 gènes isolés au lieu de 3, 31 composantes au lieu de 5. Le seuil est un **choix** qui change tout le réseau : une conclusion n'est solide que si elle résiste à plusieurs seuils.

3. Des données aléatoires

```
set.seed(1)
bruit <- matrix(rnorm(150 * 8), nrow = 150)
r_bruit <- cor(t(bruit))
adj_bruit <- (r_bruit > 0.8) * 1
diag(adj_bruit) <- 0
g_bruit <- graph_from_adjacency_matrix(adj_bruit, mode = "undirected")
ecount(g_bruit)
#> [1] 93
sum(degree(g_bruit) == 0)
#> [1] 44
set.seed(1)
comm_bruit <- cluster_louvain(g_bruit)
length(comm_bruit)
#> [1] 71
modularity(comm_bruit)
#> [1] 0.9025321
```

93 liens entre gènes **indépendants** : avec 8 échantillons, des corrélations > 0.8 apparaissent par hasard. Louvain trouve toujours des modules, et une modularité de 0.9, plus **élevée** que dans les vraies données : un réseau très creux, fait de petits morceaux, est forcément « modulaire ». Une forte modularité ne prouve pas que les modules sont biologiques.

4. Les poids lus comme des distances

```
distances(got, "Jon", "Daenerys")
#>     Daenerys
#> Jon        6
shortest_paths(got, "Jon", "Daenerys")$vpath[[1]]
#> + 4/127 vertices, named, from b3ab3dd:
#> [1] Jon           Brandon Stark Catelyn       Daenerys
```

Sans `weights = NA`, la « distance » est la **somme des poids** du chemin : 6 au lieu de 2 liens. Le chemin choisi passe par des personnages secondaires (liens de **faible** poids) et évite Ned, pourtant lié fortement aux deux. Aucun message.

5. Plusieurs graines

```
map_dfr(1:5, function(graine) {
  set.seed(graine)
  cl <- cluster_louvain(got)
  tibble(graine, communautes = length(cl),
         tailles = paste(sort(sizes(cl), decreasing = TRUE), collapse = " "),
         ned_avec_daenerys = membership(cl)["Ned"] == membership(cl)["Daenerys"])
})
#> # A tibble: 5 × 4
#>   graine communautes tailles            ned_avec_daenerys
#>    <int>       <int> <chr>              <lgl>
#> 1      1           7 45 24 19 17 16 4 2 FALSE
#> 2      2           7 45 24 19 17 16 4 2 FALSE
#> 3      3           7 47 23 18 17 16 4 2 FALSE
#> 4      4           5 47 28 19 17 16     FALSE
#> 5      5           7 45 24 19 17 16 4 2 FALSE
```

Le nombre et la taille des communautés **changent** selon la graine ; ici, Ned et Daenerys ne sont jamais ensemble, ce qui est une conclusion **stable**. C'est ce genre de vérification qui distingue un résultat d'un hasard d'exécution.

6. `components()` masqué

```
suppressPackageStartupMessages(library(DESeq2))
find("components")
#> [1] "package:generics" "package:igraph"
components(g_co)
#> Error in UseMethod("components"): no applicable method for 'components' applied to an
igraph::components(g_co)$no
#> [1] 5
```

Au chargement, R affiche `The following objects are masked from 'package:igraph': components, union` (le masquage vient du package `generics`, chargé par `BiocGenerics`, lui-même chargé par DESeq2). `components()` désigne alors une fonction **générique** qui ne connaît pas les objets igraph : `no applicable`

`method for 'components' applied to an object of class "igraph"`. L'erreur est visible ; la solution : `igraph::components()`.

Pour aller plus loin

Interpréter un module

```
library(clusterProfiler)
library(org.Hs.eg.db)
res <- read_tsv("/home/public/enrichissement/airway_deseq2.tsv")
mod_induit <- as.integer(names(which.max(table(membership(modules)[coexpr$statut == "indu
genes_module <- coexpr$gene[membership(modules) == mod_induit]
ora_module <- enrichGO(gene = genes_module, universe = res$gene[!is.na(res$padj)],
                       OrgDb = org.Hs.eg.db, keyType = "ENSEMBL", ont = "BP", readable =
as_tibble(ora_module) |> dplyr::select(Description, GeneRatio, p.adjust) |> head(10)
```

L'univers : tous les gènes **testés** d'`airway`. Les 150 gènes du réseau ne conviennent pas : ils ont été choisis parce qu'ils varient beaucoup, ce qui biaise toute comparaison.

Une vraie distance

```
d <- 1 / E(got)$weight
shortest_paths(got, "Jon", "Daenerys", weights = d)$vpath[[1]]
#> + 5/127 vertices, named, from b3ab3dd:
#> [1] Jon      Ned      Robert   Viserys  Daenerys
sort(betweenness(got, weights = d), decreasing = TRUE)[1:5]
#>     Ned Catelyn     Jon  Robert  Tyrion
#>    5507    2565    2101    1937    1695
```

Avec `1 / weight`, un lien fort est **court**. Le chemin Jon → Daenerys passe par Ned, Robert et Viserys : plus long en nombre de liens que par Ned seul, mais fait de relations fortes. Ned reste en tête de l'intermédiarité ; Jon y entre, Daenerys en sort. Trois réponses (`NA`, défaut, `1 / weight`) à trois questions différentes : c'est à vous de dire ce que signifie le poids.